

DBMS LAB – TASK 10 – CSE 3

PL/SQL / STORED PROGRAMS

Using TaxationDB

1. Aim

To develop medium-level database programs using Oracle PL/SQL and MySQL Stored Programs by combining procedures, functions, cursors, conditional logic, calculations, exception handling, and database updates using the TaxationDB.

2. Objectives

After completing this laboratory, students should be able to:

- Develop parameterized stored procedures.
 - Develop stored functions for reusable calculations.
 - Combine multiple SQL statements inside stored programs.
 - Process multiple records using cursors.
 - Apply conditional logic to database records.
 - Perform calculations using values retrieved from tables.
 - Update database records through stored procedures.
 - Handle invalid input and missing records.
 - Use exception handling in Oracle PL/SQL.
 - Use handlers in MySQL Stored Programs.
 - Combine procedures and functions to solve a real-world database problem.
 - Implement and compare the solution in Oracle and MySQL.
-

3. Prerequisites

Students should know:

- SELECT
- INSERT

- UPDATE
 - DELETE
 - WHERE
 - Aggregate functions
 - GROUP BY
 - Subqueries
 - Views
 - Primary key and foreign key
 - Variables
 - IF / ELSE
 - CASE
 - Loops
 - Procedures
 - Functions
 - Cursors
 - Basic exception handling
 - Oracle PL/SQL and MySQL Stored Programs
-

4. Laboratory Scenario

The Taxation Department wants to automate part of its **annual tax assessment process**.

For every taxpayer, the system should be able to:

1. Retrieve taxpayer details.
2. Calculate a tax assessment amount based on annual income.
3. Classify the taxpayer according to income.
4. Process multiple taxpayers using a cursor.
5. Handle taxpayers who do not exist.
6. Update the taxpayer's tax-related information when required.
7. Generate a simple assessment report.

For this laboratory, use the existing **TaxationDB** tables:

- Taxpayer
 - Income_Category
 - Financial_Year
 - Income_Record
-

5. Medium-Level Activities

Part A – Parameterized Tax Calculation

Task 1 – Tax Calculation Function

Create a function named:

`calculate_tax`

The function should accept an annual income and return the calculated tax amount using the following laboratory-defined rules:

Annual Income	Tax
≤ 500000	0
500001 – 1000000	10% of income above 500000
1000001 – 1500000	20% of income above 1000000 + 50000
> 1500000	30% of income above 1500000 + 150000

Implement the function in:

- Oracle PL/SQL
- MySQL Stored Programs

Test the function with at least three different income values.

Part B – Taxpayer Assessment Procedure

Task 2 – Individual Tax Assessment

Create a procedure named:

`generate_tax_assessment`

The procedure should accept:

`taxpayer_id`

It should retrieve:

- Taxpayer Name
- PAN Number
- Annual Income
- Occupation

Then calculate:

- Tax Amount
- Net Income after Tax

Display the complete assessment.

If the taxpayer does not exist, display:

Taxpayer not found

Implement using:

- Oracle `NO_DATA_FOUND`
 - MySQL `NOT FOUND` handler
-

Part C – Income Classification

Task 3 – Classification Function

Create a function named:

`income_category`

The function should accept annual income and return:

Income	Category
≤ 500000	Low Income
500001 – 1000000	Medium Income
1000001 – 1500000	High Income
> 1500000	Very High Income

Use the function in a query to display:

Taxpayer Name
Annual Income
Income Category

for all taxpayers.

Implement the solution in both Oracle and MySQL.

6. Level 1 Questions

Question 1 – Procedure with Calculation

Create a procedure named:

taxpayer_summary

It should accept a `taxpayer_id` and display:

- Taxpayer Name
- Annual Income

- Income Category
- Calculated Tax
- Net Income

Use the `income_category` and `calculate_tax` functions created earlier.

Question 2 – Income Record Processing

Create a procedure that accepts:

`taxpayer_id`

and calculates the total income recorded for that taxpayer from the `Income_Record` table.

Display:

Taxpayer Name
Total Recorded Income
Annual Income
Difference

where:

$\text{Difference} = \text{Annual Income} - \text{Total Recorded Income}$

If no income records exist for the taxpayer, display an appropriate message.

Question 3 – Conditional Assessment

Create a procedure that accepts a `taxpayer_id`.

Retrieve the taxpayer's annual income and display:

Taxpayer Name
Annual Income
Assessment Status

Use the following rules:

Income $\leq$ 500000
→ No Tax Assessment

Income > 500000 AND Income <= 1000000
→ Standard Assessment

Income > 1000000
→ High Income Assessment

7. Level 2 Questions

Question 1 – Cursor-Based Tax Assessment

The taxation department wants to process **all active taxpayers**.

Create a cursor-based program that processes each active taxpayer and displays:

Taxpayer Name
Annual Income
Income Category
Calculated Tax
Net Income

Implement using:

Oracle

- Explicit cursor
- Cursor FOR LOOP

MySQL

- Cursor
 - **NOT FOUND** handler
 - LOOP
-

Question 2 – Cursor with Conditional Processing

Create a cursor that processes all active taxpayers.

For every taxpayer:

- If income is ≤ 500000 , display **NO TAX**.
- If income is between 500001 and 1000000, calculate 10% tax on the amount above 500000.
- If income is above 1000000, calculate tax according to the rules given in Task 1.

Display:

Taxpayer Name
Annual Income
Tax Amount
Assessment Category

Question 3 – Procedure with UPDATE

Create a procedure named:

`update_taxpayer_income`

The procedure should accept:

`taxpayer_id`
`new_income`

Before updating:

1. Check whether the taxpayer exists.
2. Check whether the new income is greater than or equal to zero.
3. If both conditions are satisfied, update **annual_income**.
4. Otherwise, display an appropriate message.

Display the updated taxpayer information after successful modification.

Implement the solution in both Oracle and MySQL.

Question 4 – Exception Handling

Create a procedure that accepts a **taxpayer_id**.

The procedure should retrieve the taxpayer's annual income and calculate the tax.

Handle the following situations:

Oracle

- NO_DATA_FOUND
- OTHERS

MySQL

- NOT FOUND
- Appropriate SQL exception handling

Display meaningful messages instead of allowing the program to terminate unexpectedly.

8. Level 3 Questions

Question 1 – Complete Tax Assessment System

Develop a stored procedure named:

complete_tax_assessment

The procedure should accept:

taxpayer_id

and perform the following operations:

Step 1

Retrieve:

- Taxpayer Name
- PAN Number
- Occupation
- Annual Income

Step 2

Determine the income category.

Step 3

Calculate the tax amount.

Step 4

Calculate:

Net Income = Annual Income - Tax

Step 5

Display the assessment report:

TAX ASSESSMENT REPORT

Taxpayer Name :
PAN Number :
Occupation :
Annual Income :
Income Category:
Tax Amount :
Net Income :

Step 6

Handle the case where the taxpayer does not exist.

Implement the complete solution in:

- Oracle PL/SQL
- MySQL Stored Programs

Question 2 – Income Record Cursor Processing

Create a cursor-based program to process all records from `Income_Record`.

For every record, display:

Income ID
Taxpayer ID
Income Source
Category

Amount
Received Date
Financial Year

Additionally, classify each income record based on its amount:

Amount $\leq$ 50000
→ Small Income Record

50001 – 200000
→ Medium Income Record

Above 200000
→ Large Income Record

Implement using:

- Oracle cursor
- MySQL cursor with **NOT FOUND** handler

Question 3 – Combined Function and Cursor

Create:

`calculate_tax(income)`

and use it inside a cursor-based program that processes all active taxpayers.

The final output should contain:

Taxpayer Name
Annual Income
Income Category
Tax Amount
Net Income

The program should process every active taxpayer automatically.

9. Challenge Question

Taxpayer Assessment Summary

Develop a complete stored program that generates a summary for all active taxpayers.

Display:

Taxpayer Name
Annual Income
Tax Amount
Income Category
Assessment Status

At the end, display:

Total Taxpayers Processed
Total Income
Total Tax

Use:

- Cursor
- Function
- Conditional statements
- Variables
- Exception/handler
- Aggregate variables

Implement the solution in Oracle PL/SQL.

Then identify the major syntax differences required to implement the same logic in MySQL
Stored Programs.

10. Expected Learning Outcome

After completing this laboratory, students should be able to develop a medium-level stored-program solution that combines:

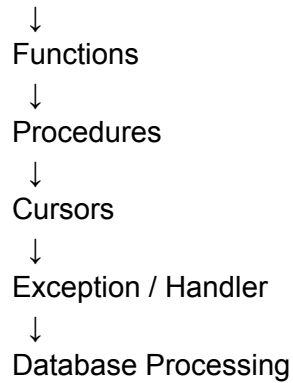
SQL



Variables



Conditional Logic



The student should be able to solve a complete real-world database-processing problem rather than writing isolated PL/SQL statements.

11. Oracle vs MySQL Implementation

Students should record the major differences observed while implementing the programs.

Concept	Oracle PL/SQL	MySQL Stored Programs
Procedure	CREATE OR REPLACE PROCEDURE	CREATE PROCEDURE
Function	CREATE OR REPLACE FUNCTION	CREATE FUNCTION
Variable assignment	<code>:=</code>	<code>SET</code>
Output	<code>DBMS_OUTPUT.PUT_LINE ()</code>	<code>SELECT</code>

Conditional	IF / ELSIF / ELSE	IF / ELSEIF / ELSE
Exception	EXCEPTION	HANDLER
No record	NO_DATA_FOUND	NOT FOUND
Cursor	CURSOR	CURSOR
Procedure execution	EXEC / PL/SQL block	CALL
Delimiter	/	DELIMITER

12. GitHub Submission

Submit the completed work in the following structure:

StudentID-repo

```

|
└── Week-10-Lab
    |
    ├── week10_plsql_oracle.sql
    |
    └── week10_stored_programs_mysql.sql

```